Problem : org.hibernate.LazyInitializationException

WHY ?

JPA/Hibernate follows default fetching policies for different types of associations
one-to-one : EAGER
one-to-many : LAZY
many-to-one : EAGER
many-to-many : LAZY

one-to-many : LAZY
Meaning : If you try to fetch details of one side(eg : Category) , will it fetch auto details of many side(blog posts) ?

NO (i.e select query will be fired only on categories table)
Why ? : for better performance

When will hibernate throw org.hibernate.LazyInitializationException ?

Any time you are trying to access un-fetched data from DB(represented by a proxy) , in a detached manner(outside the session scope)

Triggers : one-to-many
many-many
session's load method

Hibernate uses 3rd party support - ByteBuddy JAR , for the genration of dynamic proxies.

un fetched data : i.e blog post list in Category obj : represented by : proxy (substitution) : proxy object representing a collection
proxy => un fetched data from DB

Solutions
1. Change the fetching policy of hibernate for one-to-many to : EAGER
eg :

```
@OneToMany(mappedBy = "chosenCategory",cascade =
CascadeType.ALL,orphanRemoval=true,fetch=FetchType.EAGER)
 private List<BlogPost> posts=new ArrayList<>();
```

Is it recommneded soln : NO (since even if you just want to access one side details , hibernate will fire query on many side) --will lead to worst performance.
Use Case : when the size of many is small

OR
2.

```
 @OneToMany(mappedBy = "chosenCategory",cascade = CascadeType.ALL)
 private List<BlogPost> posts=new ArrayList<>();
```

Solution : Simply access the size of the collection within session scope : This soln will be applied in DAO layer
Dis Adv : Hibernate fires multiple queries to get the complete details

OR
3. Most recommended soln to avoid select n+1 issue :
How to fetch the complete details , in a single join query ?

Using "join fetch" keyword in JPQL
String jpql ="select c from Category c join fetch c.posts where c.categoryName=:nm"

Hibernate will fire SINGLE INNER JOIN query to fetch category n blog post details (not resulting in LazyInitializationException)